ANALÍTICA PARA BIG DATA

26 de fevereiro de 2026 13:10

Big Data – conjunto de dados **superior a capacidade de processamento tradicional** (Terabytes+)

Os 5 V's do Big Data

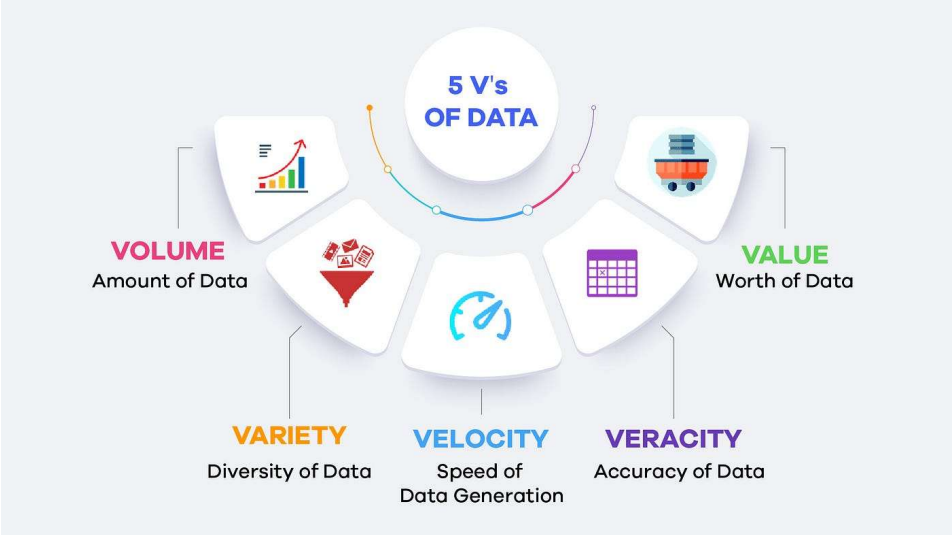
Volume: a escala pura dos dados

Velocidade: a **rapidez** com que os dados chegam e são processados

Variedade: evoluímos de tabelas para **vídeo, áudio, JSON etc**

Veracidade: a capacidade de limpar e confiar no dado, **qualidade**

Valor: os outros 4 V's não servem se não tirarmos lucro ou insights úteis.



Base de Dados Relacionais (RDBMS): ordem absoluta, se a tabela foi definida para receber números, não aceitará texto. A normalização é o processo de dividir dados em várias tabelas conectadas para evitar repetições e garantir eficiência de armazenamento.

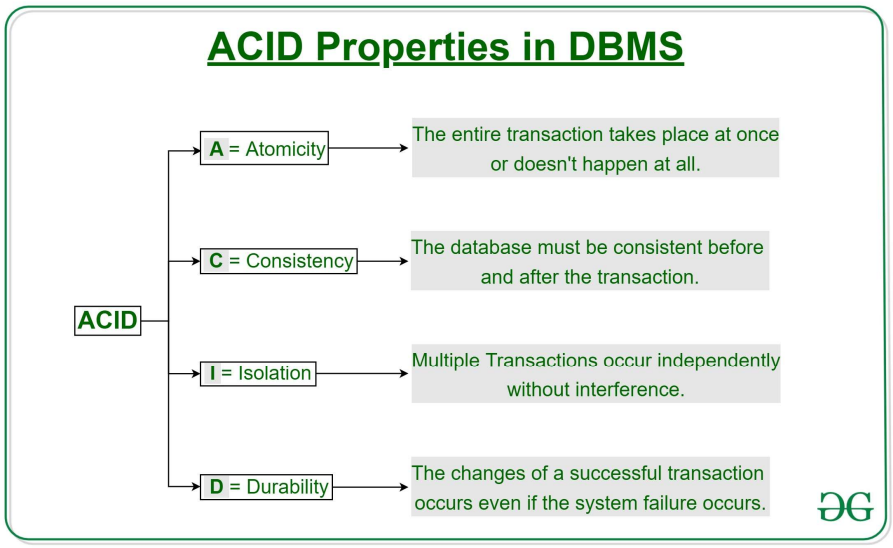
ACID

Atomicidade: tudo ou nada (**transação completa ou falha total**).

Consistência: o sistema respeita todas as **regras de integridade**.

Isolamento: **transações simultâneas não interferem** umas nas outras.

Durabilidade: **Dados** gravados **persistem mesmo após falha** de energia.



--

NoSQL (Not Only SQL): Quando a web cresceu, as **RDBMS falharam em escala**, a compra de um **servidor maior tornou-se impossível ou demasiada cara**, logo o NoSQL não substituí o SQL, mas complementa. **Abdicamos da estrutura rígida para ganhar escalabilidade horizontal**.

Teorema CAP – num sistema distribuído (Cloud), a rede vai falhar eventualmente, por isso a **Tolerância a Partição (P)** é obrigatória. E só é possível garantir 2 das 3 propriedades, logo:

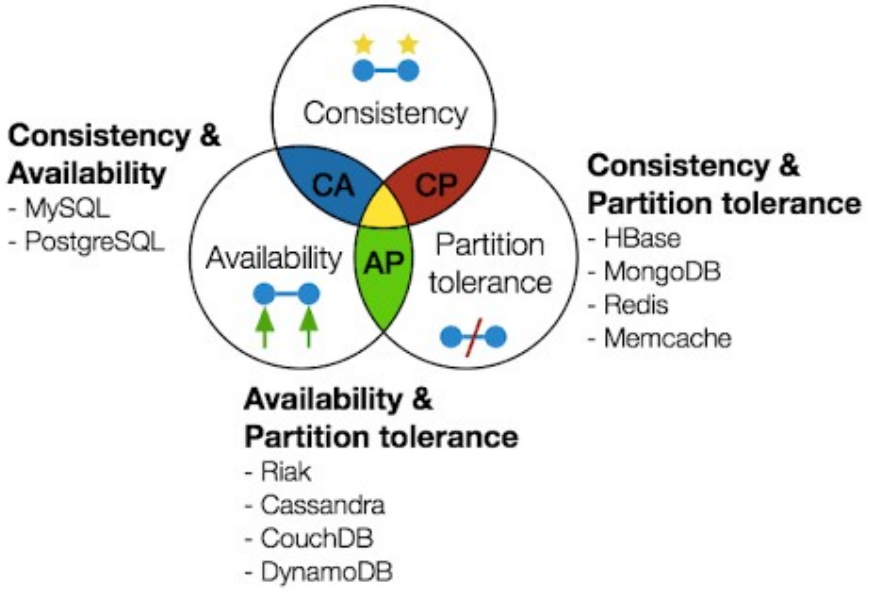
- **Sistema CP**: se a rede falha, o sistema bloqueia para garantir que não há erros de dados (ex: MongoDB)

- **Sistema AP** : se a rede falha, o sistema continua a responder, logo sacrifica a consistência.

C (Consistency) - todos veem os mesmos dados ao mesmo tempo.

A (Availability) - sistema responde sempre.

P (Partition Tolerance) - sistema funciona com falhas de redes.



-

4 Tipos de Bases de Dados NoSQL

Key-Value: dicionário gigante. Ótimo para carrinhos de compras ou sessões (Redis)

Documentos: guarda informação flexível, permite mudar campos sem parar a base (MongoDB) (JSON/XML)

Colunas: inverte a lógica(guarda por coluna), rápido para analíse. (Cassandra)

Grafos: conexões, ideal para redes sociais e recomendações. (Neo4j)

Por vezes, os dados encontram-se espalhados e não queremos mover tudo para um sítio só, usamos motores distribuídos que vão trazer os dados onde eles estão. Ferramentas como o Spark SQL ou Presto permitem escrever uma query SQL que consulta petabytes de dados distribuídos em segundos, agindo como o "cérebro" que processa a informação guardada no "corpo" (storage). No Big Data, separamos o dado da ferramente, usamos formatos abertos como Parquet ou Avro que são altamente comprimidos e eficientes para leitura rápida. O

ecossistema gira em torno de tecnologias Open Source (Apache) e soluções Cloud (AWS), que permitem pipelines que guardam streaming em tempo real e guardam tudo em "Data Lakes" (S3) para análise posterior.

- **O acesso a estes volumes massivos de dados foi um dos grandes impulsionadores para o fim do "Inverno da Inteligência Artificial". Os algoritmos de Machine Learning precisam de exemplos para aprender. O Big Dara forneceu o "combustível" necessário para treinar as IAs modernas. Passámos de escassez de dados para a abundância, e assim revolucionar a ciência de dados atual.**

-

Cloud Computing – acesso e a pedido a um conjunto partilhado de recursos informáticos configuráveis (redes ou servidores).

Níveis de Abstração

1º Hardware

2º Virtualização

3º Uso Utilizador

1º e o **2º** comunicam por Web Service

Tipos de Cloud

Pública: por uma organização e aberta ao uso do público

Privada: uso interno de uma única organização

Híbrida: combinação de pública com privada

Princípios

Utility – fornecer o serviço e gerir os custos e o utilizador não se preocupar com a infraestrutura física.

SOA (Service Oriented) - Coleção de serviço (Web Services, como SOAP ou RESTfull) que comunicam entre si através de uma rede.

SLA (Serviço Level) – contrato que garante a Qualidade de Serviço e com regras de segurança.

Virtualização - recursos físicos em Máquinas Virtuais, permitindo partilha de hardware e maior segurança.

Características Principais

Escalabilidade - adaptam-se dinamicamente (aumentam ou diminuem) consoante a procura real, evitando os problemas tradicionais de subdimensionamente (perda de clientes) ou sobredimensionamento (desperdício de dinheiro).

Disponibilidade – foco em redundância (evitar pontos únicos de falha) e recuperação de desastres.

Performance – uso de computação paralela, load balancing e agendamento de tarefas.

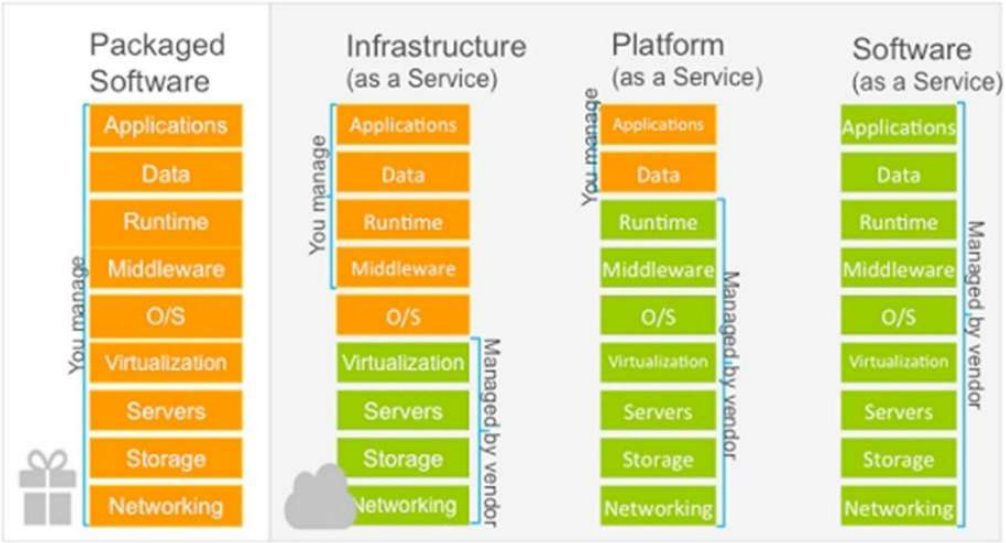
Acessibilidade – acesso a partir de qualquer lugar e dispositivo.

Modelos Cloud - Níveis

1º IaaS – fornece recursos fundamentais como computação, armazenamento e redes (Amazon EC2)

2º PaaS – oferece plataformas para implementar aplicações sem estar preocupado com a infraestrutura subjacente (AWS Elastic Beanstalk)

3º SaaS – software completo executado na nuvem, pronto a usar pelo consumidor final (Microsoft Office 365)



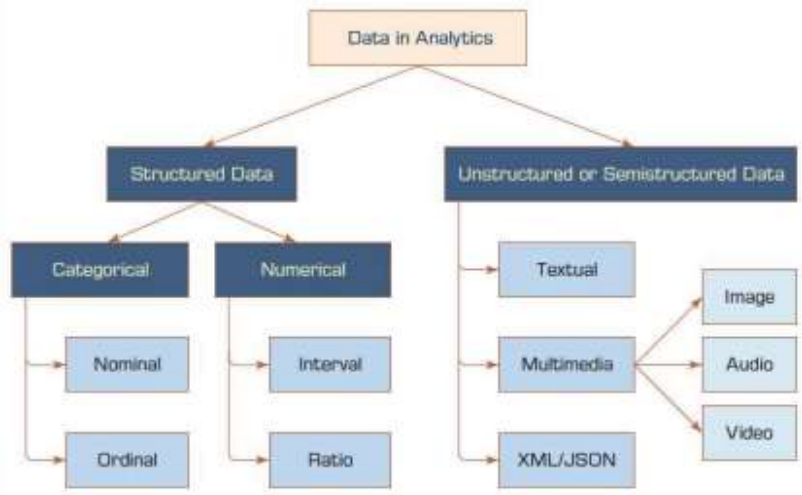
Escalar Horizontal – se precisar de aumentar a capacidade ou disponibilidade metemos outro servidor.

Escalar Vertical – ao servidor aumentamos a memoria, por exemplo.

Taxonomia dos Dados – ao nível mais alto, os dados podem ser classificados em **estruturados** ou **não estruturados**.

Estruturados - usados por algoritmos de data mining (extração de dados) e machine learning

Não Estruturados – usados em text mining (extração de texto) e web mining, ou para alimentar algoritmos de aprendizagem.



Categóricos - Qualitativos : consistem em rótulos associados a uma classe de valores numa variável

Nominal: Não ordenados (Ex: Solteiro/Casado/Divorciado)
Ordinal: Ordenado (Ex: Baixo(1) ; Médio(2) ; Alto(3))

Dados Numéricos - Quantitativos : valores numéricos, inteiros ou reais, que representam um salário, idade, número de filhos etc

Intervalar: (Ex: Temperatura, Credit Score(300-850))
Razão (Ratio) : (Ex: peso, comprimento)

Alguns algoritmos têm problemas com **dados categóricos** -> converter os valores para **numéricos.**
Valores Intervalares são um problema devido à **falta de linearidade**.
Dados não estruturados podem ser convertidos para alguma **forma categórica** ou **numérica** para serem processados.

Operação	Nominal	Ordinal	Intervalar	Razão
Distribuição de Frequências	Sim	Sim	Sim	Sim
Mediana e Percentis	Não	Sim	Sim	Sim
Adicionar ou Subtrair	Não	Não	Sim	Sim
Média e Desvio Padrão	Não	Não	Sim	Sim
Rácios e Coeficiente de Variação	Não	Não	Não	Sim

Dados semi-estruturados – MS Excel, CSV , JSON, XML
Dados não estruturados - vídeo, imagnes, texto

! Os **dados críticos** estão em bases de dados relacionais como dados estruturados, mas a maior parte dos dados das organizações também está nestes formatos. Podem atingir grandes volumes e devem ser usados em processos analíticos, embora não estejam otimizados para isso.

Formatos de Armazenamento de Big Data

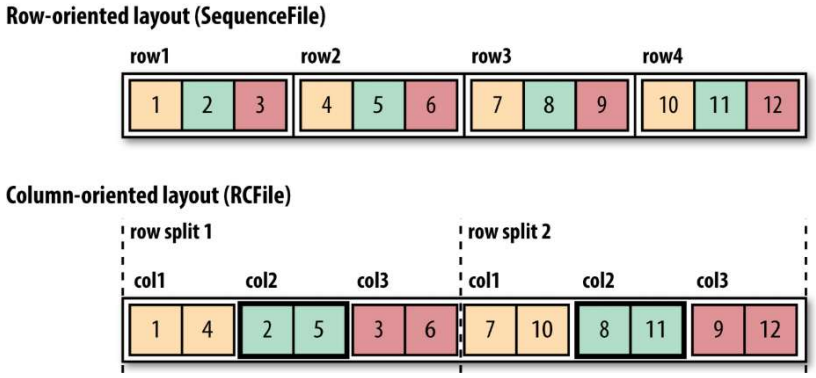
CSV – formato baseado em texto para estruturas tabulares. Colunas separadas por delimitador. Rápido a escrever e lento a ler linhas aleatórias.

```
id,nome,idade,cidade
1,Ana Silva,28,Porto
2,João Sousa,35,Lisboa
```

JSON – Formato baseado em texto para estruturas tabulares. Rápido a escrever, lento a ler linhas aleatórias.

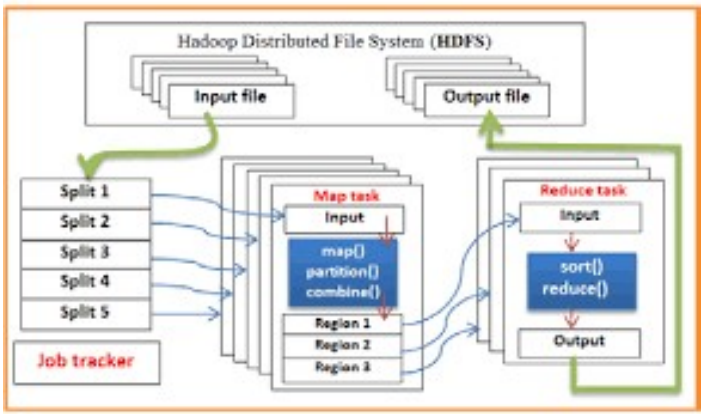
```
[
  {
    "id": 1,
    "nome": "Ana Silva",
    "idade": 28,
    "cidade": "Porto"
  },
  {
    "id": 2,
    "nome": "João Sousa",
    "idade": 35,
    "cidade": "Lisboa"
  }
]
```

Sequence File/ RC File – Ficheiros planos (flat files) que consistem em pares chave/valor binários. Extensivamente usados em MapReduce



Ecossistema e Módulos do Hadoop

Hadoop Framework - biblioteca de software (Apache) que permite o processamento distribuído de grandes volumes de dados através de clusters de computadores sendo tolerantes a falhas. Ex: em vez de um único supercomputador muito caro, liga-se milhares de computadores comuns para processar terabytes, se um falhar, o sistema continua a funcionar.

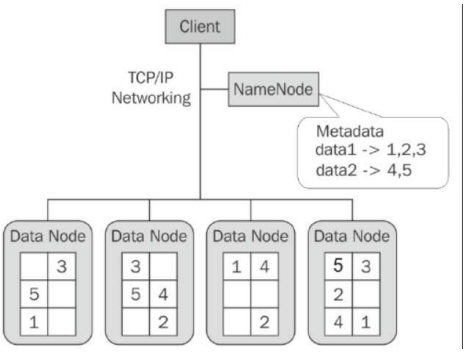


HDFS (Hadoop Distributed File System) - o sistema de ficheiros distribuído do Hadoop. É responsável por guardar os dados, dividindo-os e espalhando-os pelos vários computadores do cluster.

YARN (Yet Another Resource Negotiator) - o gestor de recursos e agendamento de tarefas do Hadoop, basicamente funcionada como um "gestor de tráfego", decidi que computadores (nós) do cluster têm memória e CPU disponíveis para executar essa tarefa.

Hadoop MapReduce (Módulo) - o sistema baseado no YARN que permite fazer o processamento paralelo da Big Data.

Arquitetura e Estratégias de Armazenamento



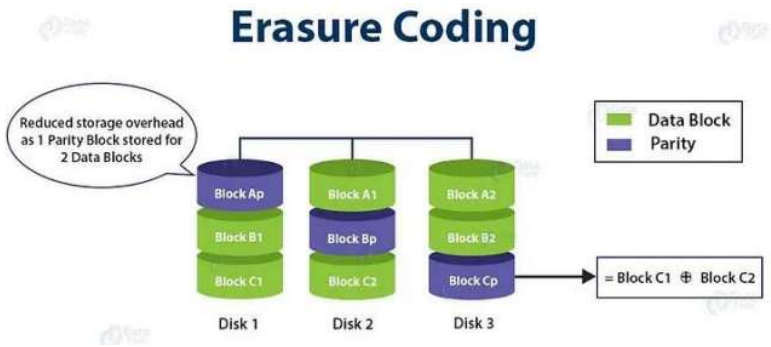
Name Node : **o nó (computador) principal no HDFS**, funciona como o "cérebro" do sistema de ficheiros, pois sabe onde cada pedaço de dado está guardado, mas não guarda os dados em si, ele diz ao sistema "o bloco 1 do ficheiro está no Servidor A e o bloco 2 está no Servidor B".

Data Nodes - os nós de trabalho onde os dados reais (os blocos) são fisicamente guardados e onde o processamento acontece, são as prateleiras da biblioteca que guardam os livros (dados).

- **Block Replication (Replicação de Blocos)** - técnica de tolerância a

falhas tradicional do HDFS onde várias cópias exatas do mesmo bloco de dados são guardadas em Data Nodes diferentes, ex: se um servidor arder, os dados não se perdem porque há cópias noutros servidores (overhead de armazenamento de 200%)

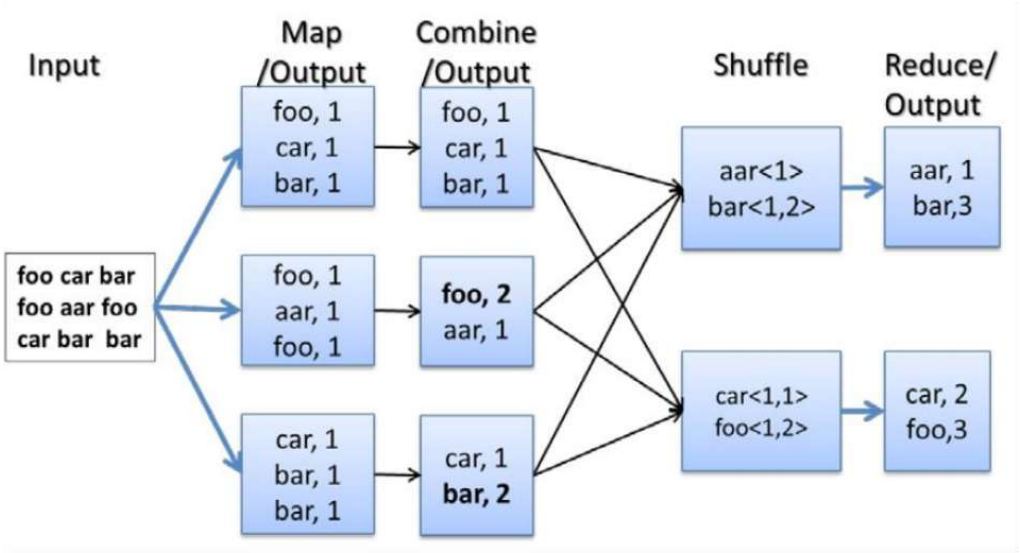
- **Erasure Coding (Codificação de Apagamento)** - alternativa à replicação, calcula "blocos de paridade" em vez de cópias exatas, poupa espaço de armazenamento, mas exigi mais processador (CPU) para recuperar dados, ex: se um bloco de dados for corrompido, o sistema faz uma equação matemátcia usando a paridade para o reconstruir, ideal para dados que não são acedidos com frequência.



- **Data Locality (Localidade dos Dados)** - o princípio de enviar o programa (processamento) para o computador onde os dados já estão fisicamente guardados, em vez de transferir os dados pela rede, ex em vez de transferir um ficheiro de 50GB pela rede para o processar noutro servidor (muito lento), o Hadoop envia o pequeno script de código diretamente para o servidor onde o ficheiro de 50GB já vive.

Modelo de Processamento

MapReduce (Modelo de Programação) - modelo que divide a execução de um trabalho em duas fases principais: **Map** (filtragem e divisão) e **Reduce** (agregação e resumo) ex: contar palavras num livro, a fase Map isola as palavras em listas separadas ("olá": 1, "mundo": 1), a fase Reduce soma essas listas para dar o resultado final ("olá" : 5, "mundo": 3).

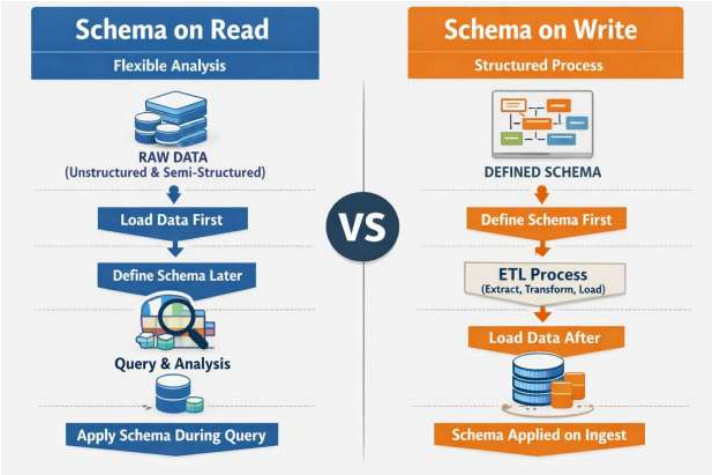


Data Lake vs Data Warehouse

Data Lake (Lago de Dados) - abordagem de armazenamento onde todos os dados são guardados no seu **formato bruto (raw)**, sem serem transformados previamente, mais barato e flexível, despejar relatórios em PDF, dados estruturados de vendas de um website para o Hadoop, sem tratamento para análises futuras

Schema on Read (Esquema na Leitura) - característica dos **Data Lakes** onde a estrutura (o esquema) dos dados só é definida ou aplicada no momento em que alguém tenta ler ou processar esses dados, basicamente o programa decide que colunas quer extrar de um ficheiro de texto, o código em Spark para fazer o relatório.

Schema on Write (Esquema na Escrita) - abordagem tradicional dos **Data Warehouse**, que exige um processo **ETL** (Extração, Transformação e Load, os dados têm de ser limpos e formatados , ex, a data do aniversario de um funcionário tem de estar como o formato correto, senão pode ser descartado.

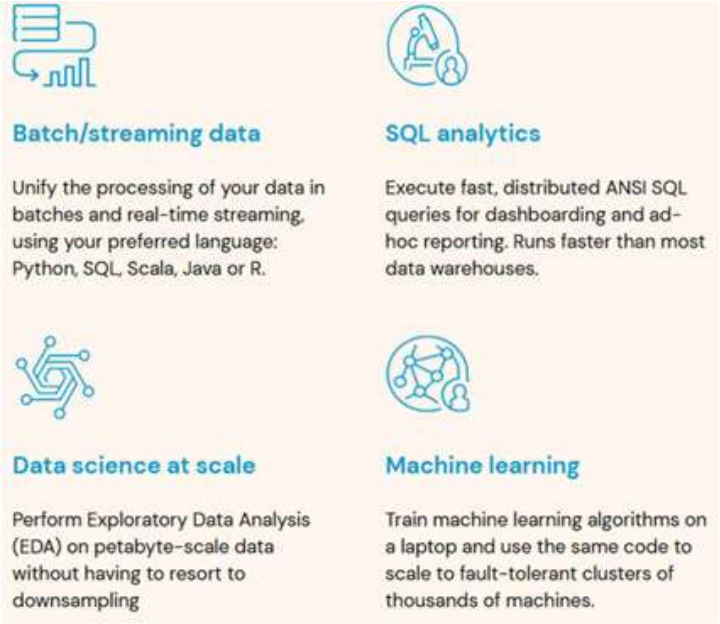


Apache Spark - motor de processamento unificado e multi-linguagem desenhado para Engenharia de Dados, Ciência de Dados e Machine Learning.

Vantagem : processa os **dados em memória (RAM)**, mais rápido que o Hadoop MapReduce (processa no disco) e tem a **programação mais simples**.

Desvantagem : soluções em Spark são **mais caras** devido ao custo da **memória RAM** em comparação com o armazenamento em disco

Filosofia : é <u>estritamente um motor de computação, não de armazenamento</u>, processa dados extraídos de outras fontes (HDFS , Amazon S3, Azure …)



Linguagens e Estruturas de Dados

O **Spark** suporta **Scala** (linguagem nativa) , Java, Python (PySpark), R e SQL.

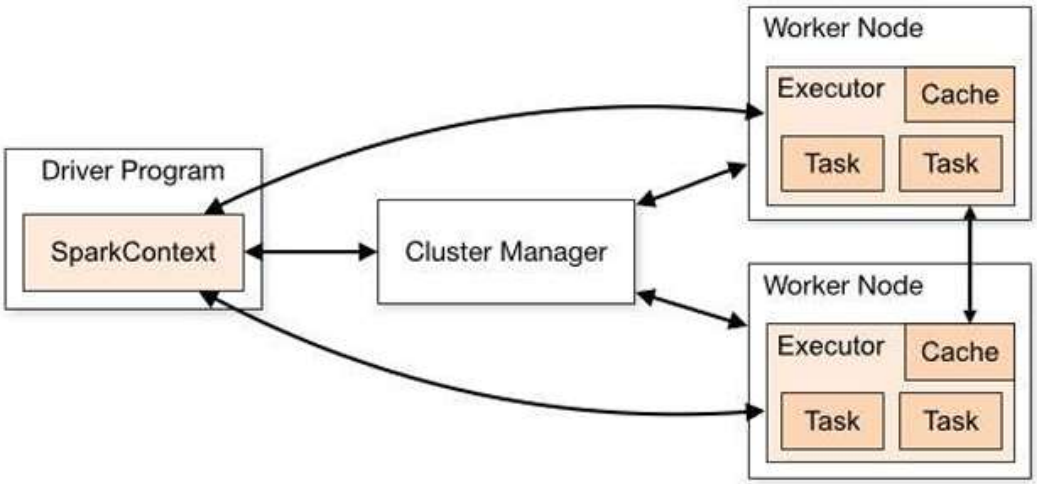
Para manipular os dados usamos as seguintes abstrações:

- **RDDs (Resilient Distributed Datasets)** - a API original de baixo nível, são coleções de dados imutáveis e distríbuidas, devem ser usados para **dados não estruturados** (como texto) ou quando precisas de um controlo muito granular/baixo nível sobre as transformações.

- **DataFrames e Datasets** : são as APIs estruturadas modernas, um DataFrame organiza os dados em colunas nomeadas (muito semelhante a uma tabela numa base de dados relacional ou um DataFrame em Pandas/R), são a escolha padrão por serem otimizados e fáceis de usar.

Arquitetura, Execução e Particionamento

O Spark pode correr **localmente (recursos limitados, ideal para testes)** ou num **Cluster (escalável**, gerido por YARN, Standalone, etc). Para processar em paralelo de forma eficiente, o Spark divide os dados em Partições.



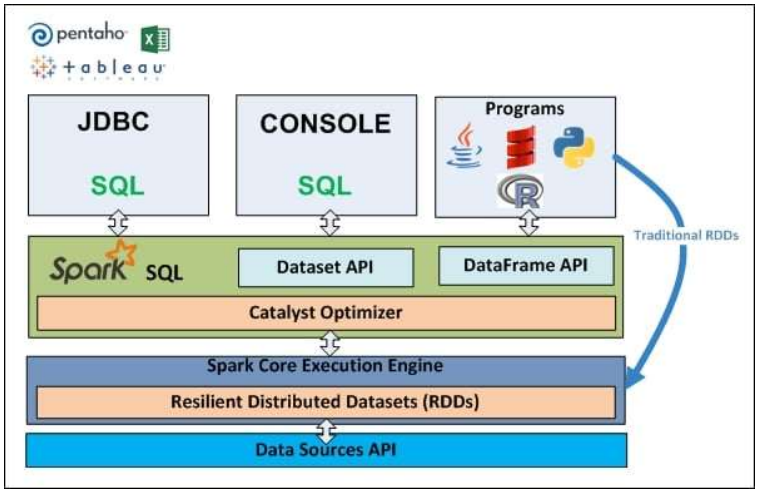
Como gerir partições?

- **repartition()** : para **aumentar ou diminuir o número de partições**, envolve um **Full Shuffle** (reorganização pesada e custosa de todos os dados pela rede), é útil quando precisas de distribuir os dados de forma perfeitamente equilibrada.

- **coalesce()** : usado apenas para **diminuir o número de partições**, é muito mais eficiente porque evita o shuffle total, minimizando a movimentação de dados pela rede.

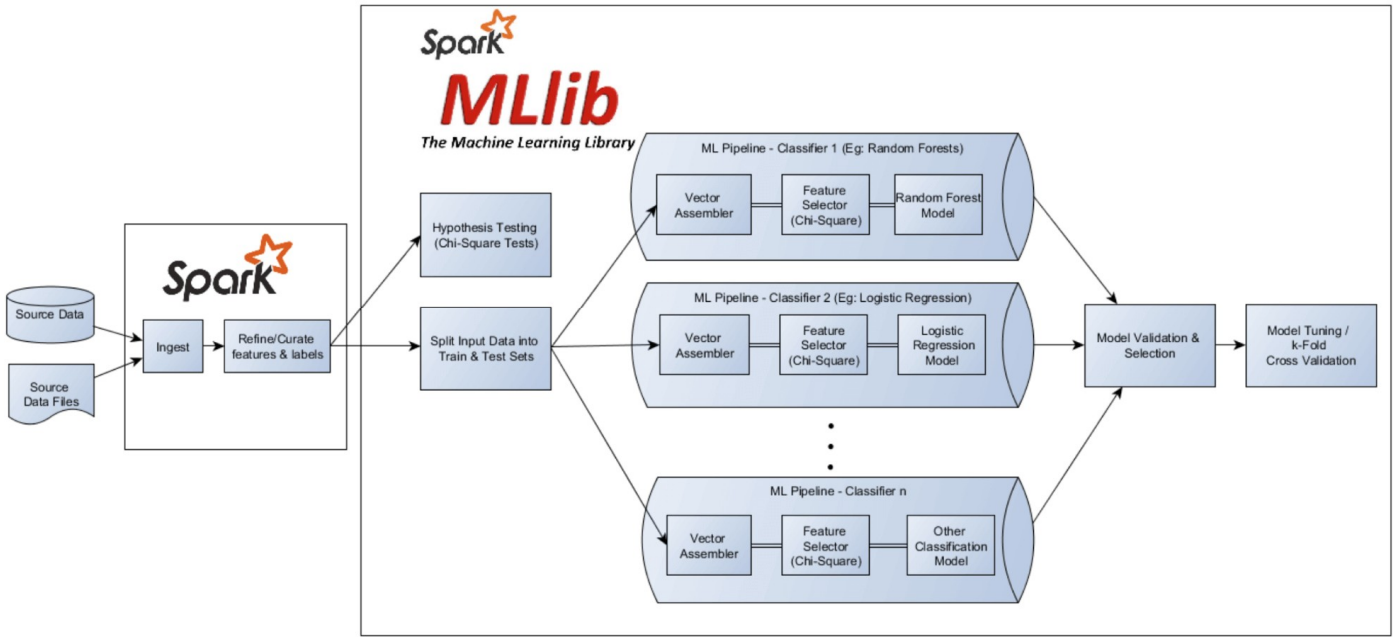
Módulos Principais do Ecossistema Spark

A. **Spark SQL** - permite **executar queries de SQL tradicional diretamente sobre ficheiros** (ex: Parquet) , **DataFrames** ou **bases de dados** conectadas (como Hive)



A. **Machine Learning Library (Mllib)** - foca-se em tornar o **ML prático e escalável**, a API moderna utiliza **DataFrames** e baseia-se no conceito de **Pipelines** (inspirado no scikit-learn)

- **DataFrame** : guarda os dados, features e labels
- **Transformer** : algoritmo que transforma um DataFrame noutro (ex: aplicar o modelo e devolver previsões)
- **Estimator** : algoritmo que treina sobre um DataFrame para produzir um modelo (Transformer).
- **Sistemas de Recomendação (ALS)** : o Spark usa o algoritmo Alternating Least Squares para filtragem colaborativa, suportando feedback explícito (avaliações claras de utilizadores) e implícito (cliques, visualizações).



A. **GraphX (Análise de Grafos)**

Módulo para processamento de grafos e redes sociais.

- **Conceitos** : baseia-se em Vértices (nós, como "pessoas") e Arestas (Edges, que são as ligações entre nós).
- **Algoritmos comuns** : PageRank (mede a importância de um nó na

rede), Connected Components (agrupa nós conectados, útil para clusters) e Triangle Counting.

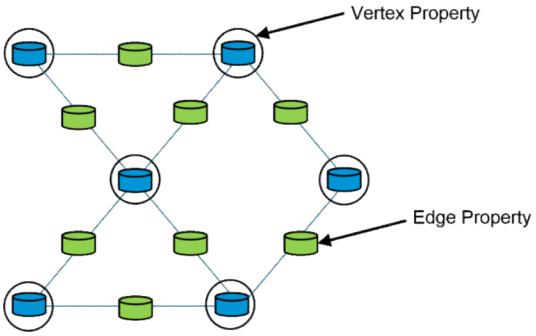


Figure: *Property Graph*

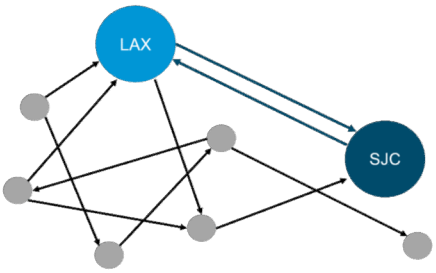


Figure: *An example of property graph*

A. Spark Streaming

Tem duas formas de lidar com fluxos de dados contínuos (ex: Kafka, Sockets):

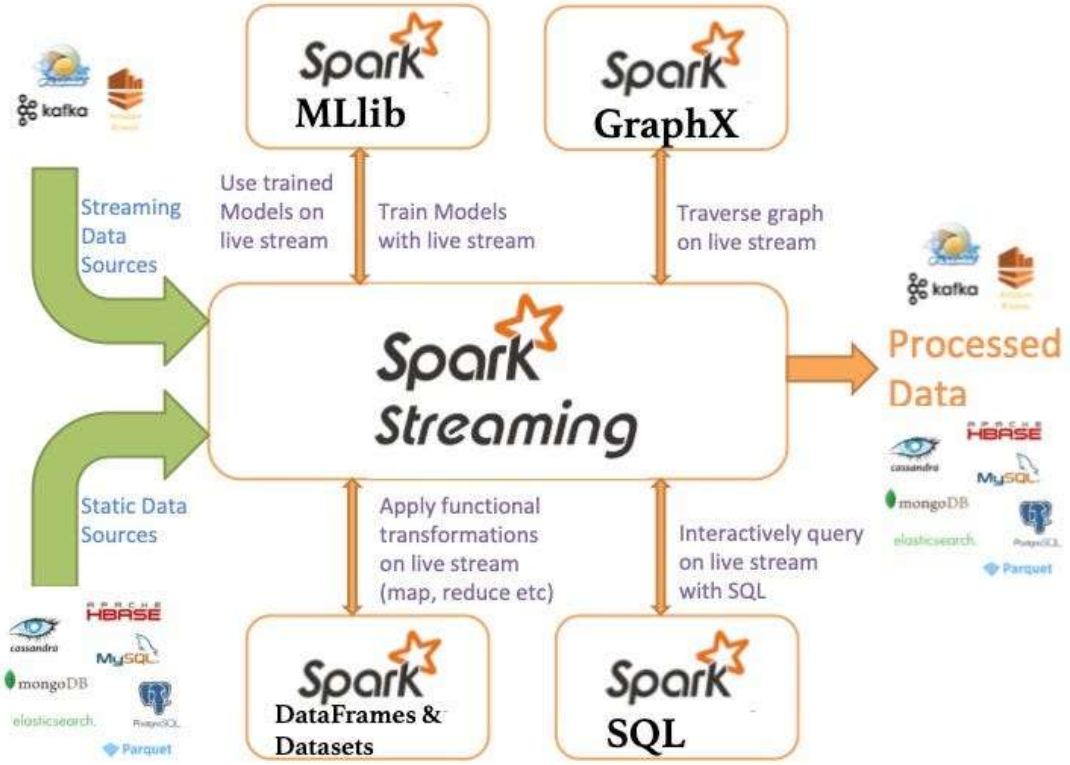
- **DStreams (API Legada)** - trabalha através de micro-batches (pequenos lotes temporais), trata o fluxo contínuo como uma série de RDDs discretos.

- **Structured Streaming (Atual)** - muito mais poderoso e simples, trata um fluxo de dados ao vivo como uma tabela que cresce continuamente (novas linhas vão sendo adicionadas), pode gerar outputs em três modos:

    - **Complete** - escreve toda a tabela atualizada

    - **Append** - escreve apenas as linhas novas

    - **Update** - escreve apenas as linhas que foram alteradas desde a última atualização


Spark
MLlib
Spark
GraphX
Use trained
Models on
live stream
Train Models
with live stream
Traverse graph
on live stream
Streaming
Data
Sources
Spark
Streaming
Processed
Data
Static Data
Sources
Apply functional
transformations
on live stream
(map, reduce etc)
Interactively query
on live stream
with SQL
Spark
DataFrames &
Datasets
Spark
SQL
kafka
HBASE
cassandra
MySQL
mongoDB
elasticsearch
PostgreSQL
Parquet